



III Year I Semester	OBJECT ORIENTED ANALYSIS AND DESIGN (PROFESSIONAL ELECTIVE-I)	L	T	P	C
		3	0	0	3

Course Objectives: The main objective is the students to

1. Become familiar with all phases of OOAD.
2. Master the main features of the UML.
3. Master the main concepts of Object Technologies and how to apply them at work and develop the ability to analyze and solve challenging problem in various domains.

Course Outcomes: The students should be able to:

1. Select the basic elements of modeling such as Things, Relationships and Diagrams depending on the views of UML Architecture and SDLC.
2. Apply basic and Advanced Structural Modeling Concepts for designing real time applications.
3. Design Class and Object Diagrams that represent Static Aspects of a Software System.
4. Analyze Dynamic Aspects of a Software System using Use Case, Interaction and Activity Diagrams.
5. Apply techniques of State Chart Diagrams and Implementation Diagrams to model behavioral aspects and Runtime environment of Software Systems.

UNIT I:

Introduction: The Structure of Complex systems, The Inherent Complexity of Software, Attributes of Complex System, Organized and Disorganized Complexity, Bringing Order to Chaos, Designing Complex Systems. **Case Study:** System Architecture: Satellite-Based Navigation

UNIT II:

Introduction to UML: Importance of modeling, principles of modeling, object oriented modeling, conceptual model of the UML, Architecture, and Software Development Life Cycle. **Basic Structural Modeling:** Classes, Relationships, common Mechanisms, and diagrams. **Case Study:** Control System: Traffic Management.

UNIT III:

Class & Object Diagrams: Terms, concepts, modeling techniques for Class & Object Diagrams. **Advanced Structural Modeling:** Advanced classes, advanced relationships, Interfaces, Types and Roles, Packages. **Case Study:** AI: Cryptanalysis.

UNIT IV:

Basic Behavioral Modeling-I: Interactions, Interaction diagrams Use cases, Use case Diagrams, Activity Diagrams. **Case Study:** Web Application: Vacation Tracking System

11



UNIT V:

Advanced Behavioral Modeling: Events and signals, state machines, processes and Threads, time and space, state chart diagrams. **Architectural Modeling:** Component, Deployment, Component diagrams and Deployment diagrams. **Case Study:** Weather Forecasting

Text Books:

- i. Grady BOOCH, Robert A. Maksimchuk, Michael W. ENGLE, Bobbi J. Young, Jim Conallen, Kellia Houston, "Object- Oriented Analysis and Design with Applications", 3rd edition, 2013, PEARSON.
- ii. Grady Booch, James Rumbaugh, Ivar Jacobson: The Unified Modeling Language User Guide, Pearson Education.

Reference Books:

- i. Meilir Page-Jones: Fundamentals of Object Oriented Design in UML, Pearson Education.
- ii. Pascal Roques: Modeling Software Systems Using UML2, WILEY- Dreamtech India Pvt. Ltd.
- iii. AtulKahate: Object Oriented Analysis & Design, The McGraw-Hill Companies.
- iv. Applying UML and Patterns: An introduction to Object – Oriented Analysis and Design and Unified Process, Craig Larman, Pearson Education.



III Year I Semester	COMPILER DESIGN	L	T	P	C
		3	0	0	3

Course Objectives:

The objectives of this course is to acquire knowledge on the

1. Design of lexical analyzers, Syntax analyzers, Intermediate code generators
2. Usage of Lex/Bison tools in writing compilers
3. Issues in the code generation, code generation generation algorithms.

Course Outcomes:

The students should be able to:

1. Understand the basics of language processing and implement lexical analyzer for any language.
2. Describe the different types of parsing and implement parser for any language.
3. Explain the different intermediate code representations and use Syntax directed
4. Apply the basics of data flow analysis, optimizations, and run time environment required for handling recursive procedures.
5. Discuss the issues in the code generation and code generation algorithms.

UNIT - I:

Introduction and Lexical Analysis: Language Processors, the structure of a compiler, the science of building a compiler, phases of a compiler. Lexical Analysis: The role of the lexical analyzer, Identifying tokens, Transition diagrams for recognizing tokens, Input buffering, The lexical analyzer generator Lex, Finite automata, Conversion from regular expressions to automata, design of a lexical analyzer generator, Optimization of DFA-based pattern matchers.

UNIT - II:

Syntax Analysis: Introduction, Context-Free Grammars, BNF(Backus-Naur Form), EBNF(Extended Backus-Naur Form). Preprocessing of grammars: left recursion elimination, left factoring. Top-Down Parsing: Recursive-descent parsers, LL(1) parsers. Bottom-Up parsing: Introduction to LR parsers, Simple LR, Canonical LR, and Look ahead LR. Extending parsers to handle ambiguous grammars, Parser generators Yacc/Bison.

UNIT – III:

Syntax-Directed Translation, Semantic Analysis, Intermediate Code Generation

Syntax-Directed Definitions, Evaluation orders for SDD's, Applications of Syntax-Directed Translation, Syntax-Directed Translation Schemes, and Implementing L-Attributed SDD's.

Intermediate code generation: Variants of Syntax trees, Three-address code, Types and declarations, Type checking, Control flow, Back patching, Switch-Statements, Intermediate Code for Procedures.

UNIT - IV:

Code Optimization, Run-time Environment

Run-Time Environments: Storage organization, Activation record, Stack allocation, Access to nonlocal data on the stack, Heap management, Introduction to garbage collection, Introduction to



trace-based collection. Machine-Independent optimizations: The principal sources of optimization, Basic blocks and flow graphs, Introduction to data-flow analysis, Foundations of data-flow analysis, Constant propagation.

UNIT - V: Target Code Generation

Code Generation: Issues in the design of a Code Generator, The target language, Addresses in the target code, A simple code generator.

Machine-dependent Optimizations: Peephole optimization, Register allocation and assignment, Dynamic Programming code generation.

Text Books:

- i. Compilers: Principles, Techniques and Tools, Second Edition, Alfred V. Aho, MonicaS. Lam, Ravi Sethi, Jeffry D. Ullman, Pearson.
- ii. Compiler Construction-Principles and Practice, Kenneth C Loudon, Cengage Learning.

Reference Books:

- i. Modern Compiler Implementation in C, Andrew W Appel, Revised edition, Cambridge University Press.
- ii. The Theory and Practice of Compiler writing, J. P. Tremblay and P. G. Sorenson, TMH
- iii. lex & yacc, 2nd Edition by John Levine, Doug Brown, Tony Mason

E-resources:

- i. <https://www.edx.org/course/compilers>
- ii. <https://nptel.ac.in/courses/106/108/106108113/>

III Year I Semester	DATA WAREHOUSING & DATA MINING	L	T	P	C
		3	0	0	3

Course Objectives:

Students undergoing this course are expected to:

1. Understand the concepts of Data Warehousing and Data Mining
2. Master data mining techniques in various applications like social, scientific and environmental context.
3. Develop skill in selecting the appropriate data mining algorithm for solving practical problems.

Course Outcomes:

After completion of the course, students should be able to:

1. Understand Data Warehouse fundamentals, Data Mining concepts, principles and its functionalities
2. Preprocess the data using various Data Preprocessing Techniques for mining applications
3. Design and deploy appropriate classification techniques to solve real world problems and further be able to assess the strengths and weaknesses of various methods and algorithms to analyze their behavior.
4. Demonstrate Association analysis techniques for generating association rules from data.
5. Use different Clustering techniques to cluster data and Cluster the high dimensional data for better organization of the data

UNIT –I: Data Mining Systems and Knowledge Discovery Process:

Data Warehouse and OLAP Technology: An Overview- What Is a Data Warehouse. A Multidimensional Data Model - Need for Online Analytical Processing - OLTP V/s OLAP -OLAP Operations in Multidimensional Data Model. Data Warehouse Architecture, Data Warehouse Implementation, From Data Warehousing to Data Mining.

Need and Usage of Data Mining Technologies - Overview of Knowledge Discovery Process from Databases—What Motivated Data Mining - Why Is It Important - Data Mining Functionalities—What Kinds of Patterns Can Be Mined? Are All of the Patterns Interesting Classification of Data Mining Systems, Data Mining Task Primitives, Integration of a Data Mining System with a Database or Data Warehouse System, Major Issues in Data Mining.

UNIT–II: Data Pre-processing:

Data Exploration: Data Objects and attribute types -Statistical description of data- Descriptive Data Summarization-Data Visualization - Data similarity and dissimilarity measures.

Data Pre-processing: Why Pre-process the Data -Data Cleaning-Data Integration-Data Reduction-Data Transformation and Data Discretization.

UNIT–III: Classification:

Basic issues regarding classification and predication - General Approach to solving a classification problem- Decision Tree Classification, Attribute Selection Measures, Tree Pruning- Bayesian

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY GURJADA VIZIANAGARAM

VIZIANAGARAM – 535 003, Andhra Pradesh, India

B.Tech CSE (R23-COURSE STRUCTURE & SYLLABUS)



Classification – Rule Based Classification – Support Vector Machines.

Classification Model Evaluation and Selection - Accuracy and Error measures, Holdout, Random Sampling, Cross Validation, Bootstrap, Comparing Classifier performance using ROC Curves.

UNIT–IV: Mining Frequent Patterns and Association Rules:

Basic Concepts-Problem Definition- Market Basket Analysis- Frequent Itemsets- Closed Itemsets and Association Rules - Frequent Pattern Mining - Efficient and Scalable Frequent Itemset Mining Methods- the Apriori Algorithm for finding Frequent Itemsets Using Candidate Generation - Generating Association Rules from Frequent Itemsets - A pattern growth approach for mining Frequent Itemsets- FP-Growth Algorithm

UNIT V: Cluster Analysis:

Basics and Importance of Cluster Analysis- Clustering techniques- Different Types of Clusters-Partitioning Methods (K-Means, K Medoids) -Strengths and Weaknesses. Hierarchical Methods (Agglomerative, Divisive) - Density-Based Methods (DBSCAN, OPTICS)-

Text Books:

- i. Introduction to Data Mining: Pang-Ning Tan & Michael Steinbach, Vipin Kumar, Pearson.
- ii. Data Mining concepts and Techniques, 3/e, Jiawei Han, Michel Kamber, Elsevier.

References:

- i. Data Mining Techniques and Applications: An Introduction, Hongbo Du, Cengage Learning.
- ii. Data Mining : VikramPudi and P. Radha Krishna, Oxford.
- iii. Data Mining and Analysis - Fundamental Concepts and Algorithms; Mohammed J.Zaki, Wagner Meira, Jr, Oxford
- iv. Data Warehousing Data Mining & OLAP, Alex Berson, Stephen Smith, TMH.

E-resources:

- i. http://onlinecourses.nptel.ac.in/noc18_cs14/preview (NPTEL course by Prof.Pabitra Mitra)
- ii. http://onlinecourses.nptel.ac.in/noc17_mg24/preview (NPTEL course by Dr. Nandan Sudarshanam & Dr. Balaraman Ravindran)
- iii. http://www.saedsayad.com/data_mining_map.htm



III Year I Semester	DESIGN AND ANALYSIS OF ALGORITHMS	L	T	P	C
		3	0	0	3

Course Objectives:

The objectives of this course is to acquire knowledge on the

1. Analyze the asymptotic performance of algorithms
2. Write rigorous correctness proofs for algorithms
3. Demonstrate a familiarity with major algorithms and data structures

Course Outcomes: The students should be able to:

1. Measure the performance and calculate the Time & Space complexities of algorithms.
2. Design effective algorithms based on strategies like Decrease and Conquer, Divide and Conquer Transform and Conquer,
3. Develop efficient algorithms using Greedy method.
4. Discuss various problems suitable to Dynamic programming.
5. Find an optimal solution by applying different Backtracking and Branch & Bound technique.

UNIT - I:

Introduction: What is an Algorithm, Algorithm Specification, Pseudo code Conventions Recursive Algorithm, Performance Analysis, Space Complexity, Time Complexity, Amortized Complexity, Amortized Complexity, Asymptotic Notation, Practical Complexities' Performance Measurement.

UNIT - II:

Decrease-and-Conquer: Insertion Sort Algorithms for Generating Combinatorial Objects Decrease-by-a-Constant-Factor Algorithms Variable-Size-Decrease Algorithms

Divide and Conquer: Merge Sort, Quick Sort, Multiplication of Large Integers and Strassen's Matrix Multiplication

Transform and conquer: Pre-sorting Balanced Search Trees, Heaps and Heap sort

UNIT - III:

The Greedy Method: The General Method, Knapsack Problem, Job Sequencing with Deadlines Minimum-cost Spanning Trees, Prim's Algorithm, Kruskal's Algorithms, An Optimal Randomized Algorithm, Optimal Merge Patterns, Single Source Shortest Paths.

UNIT - IV:

Dynamic Programming: The General Method Multistage graph ,All - Pairs Shortest Paths, , String Edition, 0/1 Knapsack, Reliability Design, optimal binary search trees.

UNIT - V:

Backtracking: The General Method, The S-Queens Problem, Sum of Subsets, Graph Coloring Hamiltonian Cycles

Branch and Bound: The Method, Least cost (LC) Search, The 15-Puzzle: an Example, Control Abstraction for LC-Search, Bounding, FIFO Branch-and-Bound, LC Branch and Bound, 0/1 Knapsack problem, LC Branch-and Bound Solution, FIFO Branch-and-Bound Solution, Traveling Salesperson problem.

Text Books:

- i. Fundamentals of computer algorithms E. Horowitz S. Sahni, University Press



- ii. Introduction to the design and analysis of Algorithms Anany Levitin Pearson ,3rd edition
- iii. Introduction to Algorithms Thomas H Cormen PHILearning

Reference Books:

- i. The Design and Analysis of Computer Algorithms, Alfred V Aho John E Hopcroft Jeffrey DULLman
- ii. Algorithm Design, Jon Kleinberg, Pearson
- iii. Algorithms, by Dasgupta, Papadimitrou and Vazirani, McGraw-Hill Education, 2006.



IV Year I Semester	INTERNET OF THINGS (PROFESSIONAL ELECTIVE-IV)	L	T	P	C
		3	0	0	3

Course Objectives:

The objectives of this course is to acquire knowledge on the

1. What IoT is and how it works today and to Understand the Architectural Overview of IoT
2. To Understand the IoT Reference Architecture and Real World Design Constraints
3. To understand the various IoT Protocols.

Course Outcomes:

The students should be able to:

1. Recognize the factors that contributed to the emergence of IoT
2. Design and program IoT devices like Microcontrollers, sensors and actuators.
3. Use real IoT protocols for communication.
4. Define the infrastructure for supporting IoT deployment.
5. Design an IoT device to work with a Cloud Computing infrastructure and Transfer IoT data to the cloud and in between cloud providers.

UNIT-I: Introduction to IOT

Understanding IoT fundamentals, IOT Architecture and protocols, Various Platforms for IoT, Real time Examples of IoT , Overview of IoT components and IoT Communication Technologies, Challenges in IOT.

UNIT - II: Arduino Simulation Environment

Arduino Uno Architecture, Setup the IDE, Writing Arduino Software, Arduino Libraries, Basics of Embedded C programming for Arduino, Interfacing LED, push button and buzzer with Arduino, Interfacing Arduino with LCD.

Sensor & Actuators with Arduino

Overview of Sensors working, Analog and Digital Sensors, Interfacing of Temperature, Humidity, Motion, Light and Gas Sensor with Arduino, Interfacing of Actuators with Arduino. Interfacing of Relay Switch and Servo Motor with Arduino.

UNIT - III: Raspberry Pi Programming

Installing and **Configuring** the Raspberry Pi, Getting Started with the Raspberry Pi, Using the Pi as a Media Centre, Productivity Machine and Web Server, Remote access to the Raspberry Pi. Preparing **Raspberry Pi for IoT Projects**.

Creating the Sensor Projects, Creating the actuator Projects, Creating a IoT controller, creating a camera and working with HTTP protocol.

UNIT - IV: Basic Networking with ESP8266 WiFi module

Basics of Wireless Networking, Introduction to ESP8266 Wi-Fi Module, Various Wi-Fi library, Web server- introduction, installation, configuration, Posting sensor(s) data to web server .IoT Protocols ,M2M vs. IOT Communication Protocols.

UNIT -V: Cloud Platforms for IOT

Virtualization concepts and Cloud Architecture , Cloud computing, benefits ,Cloud services -- SaaS, PaaS, IaaS , Cloud providers & offerings ,Study of IOT Cloud platforms , Thing Speak API and MQTT , interfacing ESP8266 with Web services

**Text Books:**

- i. Simon Monk, Programming Arduino: Getting Started with Sketches, Second Edition McGraw- Hill Education
- ii. Peter Waher, Learning **Internet** of Things, Packt publishing.
- iii. Ovidiu Vermesan, Peter Friess, IoT-From Research and Innovation to Market deployment, River Publishers

Reference Books:

- i. Jan Holler, Vlasios Tsiatsis, Catherine Mulligan, Stefan Avesand, Stamatis Karnouskos, David Boyle, "From Machine-to-Machine to the **Internet** of Things: Introduction to a New Age of Intelligence", 1st Edition, Academic Press, 2014.
- ii. Peter Waher, "Learning **Internet** of Things", PACKT publishing, BIRMINGHAM – MUMBAI
- iii. Bernd Scholz-Reiter, Florian Michahelles, "Architecting the **Internet** of Things", ISBN 978-3- 642-19156-5 e-ISBN 978-3-642-19157-2, Springer.



III Year I Semester	COMPILER DESIGN LAB	L	T	P	C
		0	0	3	1.5

Course Objectives:

The objectives of this course is to acquire knowledge on the

1. Implementation of a compiler for a basic language
2. Lex/Yacc specifications for designing frontend of a compiler
3. Generate MIPS instruction set

Course Outcomes:

The students should be able to:

1. Understand the different phases of compilation and the working of compilers like gcc, clang etc
2. Implement lexical analyzer for any language
3. Implement parser for any language
4. Implement 3-address code generator for simple programming constructs
5. Implement MIPS code generator by considering simple programming constructs

List of experiments

1. Check the output of different compilers gcc, g++, clang, clang++, javac, python etc by running respective language programs with different flags. (purpose to understand preprocessor, optimizations, linker)
2. The Language called TinyCStr is described as follows
 - a) Every TinyCStr program has one or more functions and syntax of function declaration and function definition is similar to C, one function must be main.
 - b) Every TinyCStr function has zero or statements
 - c) The possible statements are declaration, assignment, conditional statements (if, else, for, while) except switch.
 - d) TinyCStr supports primitive data types of C and a string datatype
 - i. Implement a lexical analyser for TinyCStr using flex/lex
 - ii. Implement a parser for TinyCStr using bison/yacc and generate AST (Abstract Syntax Tree)
 - iii. Generate a 3-address code from the AST
 - iv. Generate MIPS instructions from 3-address code and run it on SPIM simulator
3. Write a program illustrating code optimization techniques:
 - i) Constant folding
 - ii) Copy propagation
 - iii) Common sub expression elimination
 - iv) Loop unrolling
 - v) Dead code elimination

Text Books:

- i. flex & bison by John Levine Released August 2009 Publisher(s): O'Reilly Media, Inc.
- ii. Compilers: Principles, Techniques and Tools, Second Edition, Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, Pearson.

Reference Books:

- i. LLVM Cookbook, Mayur pandey

E-resources:



III Year I Semester	DATA MINING LAB	L	T	P	C
		0	0	3	1.5

Course Objectives:

1. Practical exposure on implementation of well known data mining tasks and their effective use in discovering interesting hidden patterns from large datasets.
2. Exposure to real time data sets for analysis and prediction.
3. Focus is on the main process of data mining such as data preparation, classification, clustering, association analysis, and pattern evaluation

Software Requirements: WEKA Tool and R Programming/Python Programming

Course Outcomes: After undergoing the course students will be able to:

1. Create summary statistics for the given datasets.
2. Analyze various datasets and perform Data Pre-processing.
3. Apply various data mining algorithms on the give data set to select the appropriate one.
4. Develop skills and apply data mining tools for solving practical problems.
5. Handling a small data mining project for a given practical domain.

LIST OF EXPERIMENTS:

1. Study of WEKA tool and applying data mining techniques on following data sets in ARFF or CSV file Format.
2. Implementation / Usage of WEKA for classification of datasets such as customer's data, weather forecasting data, agricultural data etc.
3. Experiment to summarize and visualization of various datasets.
4. Experiment to demonstrate various data pre-processing techniques
5. Experiment to select prominent feature subsets of various datasets.
6. Experiment to Evaluate Information Gain of an attribute in the student database
7. Demonstration of classification rule process using j48 decision tree algorithm
8. Demonstration of classification rule process using ID3 decision tree algorithm
9. Experiment to predict the class using the Bayesian classification
10. Experiment to predict the class using the k-Nearest Neighbor classification
11. Experiment to implement weight & bias updating using the Back propagation Neural Network
12. Demonstration of clustering process using k-means algorithm
13. Demonstration of mining frequent patterns using Apriori algorithm
14. Demonstration of mining frequent patterns using FP-Growth algorithm
15. Experiment to compare the performance of various data mining algorithms on the give database.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY GURJADA VIZIANAGARAM
VIZIANAGARAM – 535 003, Andhra Pradesh, India
B.Tech CSE (R23-COURSE STRUCTURE & SYLLABUS)

III Year I Semester	FULL STACK DEVELOPMENT - 2 (SKILL ENHANCEMENT COURSE)	L	T	P	C
		0	1	2	2

Course Objectives:

The main objectives of the course are to

1. Make use of router, template engine and authentication using sessions to develop application in ExpressJS.
2. Build a single page application using RESTful APIs in ExpressJS
3. Apply router and hooks in designing ReactJS application

Course Outcomes:

After completion of the course, the students will be able to:

1. Apply core concepts of ExpressJS to build web applications
2. Design and develop web applications using ExpressJS by connecting to MongoDB with Mongoose
3. Creating reusable components using both function-based and class-based approaches.
4. Implement dynamic user interfaces in ReactJS
5. Install and configure MongoDB (including Atlas) and perform essential database operations

Experiments covering the Topics:

1. ExpressJS – Routing, HTTP Methods, Middleware, Templating, Form Data
2. ExpressJS – Cookies, Sessions, Authentication, Database, RESTful APIs
3. ReactJS – Render HTML, JSX, Components – function & Class, Props and States, Styles, Respond to Events
4. ReactJS – Conditional Rendering, Rendering Lists, React Forms, React Router, Updating the Screen
5. ReactJS – Hooks, Sharing data between Components, Applications – To-do list and Quiz
6. MongoDB – Installation, Configuration, CRUD operations, Databases, Collections and Records

Sample Experiments:

1. ExpressJS – Routing, HTTP Methods, Middleware.

- a. Write a program to define a route, Handling Routes, Route Parameters, Query Parameters and URL building.
- b. Write a program to accept data, retrieve data and delete a specified resource using http methods.
- c. Write a program to show the working of middleware.

2. ExpressJS – Templating, Form Data

- a. Write a program using templating engine.
- b. Write a program to work with form data.



3. ExpressJS – Cookies, Sessions, Authentication

- Write a program for session management using cookies and sessions.
- Write a program for user authentication.

4. ExpressJS – Database, RESTful APIs

- Write a program to connect MongoDB database using Mongoose and perform CRUD operations.
- Write a program to develop a single page application using RESTful APIs.

5. ReactJS – Render HTML, JSX, Components – function & Class

- Write a program to render HTML to a web page.
- Write a program for writing markup with JSX.
- Write a program for creating and nesting components (function and class).

6. ReactJS – Props and States, Styles, Respond to Events

- Write a program to work with props and states.
- Write a program to add styles (CSS & Sass Styling) and display data.
- Write a program for responding to events.

7. ReactJS – Conditional Rendering, Rendering Lists, React Forms

- Write a program for conditional rendering.
- Write a program for rendering lists.
- Write a program for working with different form fields using react forms.

8. ReactJS – React Router, Updating the Screen

- Write a program for routing to different pages using react router.
- Write a program for updating the screen.

9. ReactJS – Hooks, Sharing data between Components

- Write a program to understand the importance of using hooks.
- Write a program for sharing data between components.

10. MongoDB – Installation, Configuration, CRUD operations

- Install MongoDB and configure ATLAS
- Write MongoDB queries to perform CRUD operations on document using insert(), find(), update(), remove()

11. MongoDB – Databases, Collections and Records

- Write MongoDB queries to Create and drop databases and collections.
- Write MongoDB queries to work with records using find(), limit(), sort(), createIndex(), aggregate().

12. Augmented Programs: (Any 2 must be completed)

- Design a to-do list application using NodeJS and ExpressJS.



- Design a Quiz app using ReactJS.
- Complete the MongoDB certification from MongoDB University website.

Text Books:

- Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node, Vasan Subramanian, 2nd edition, APress, O'Reilly.
- Node.js in Action, Mike Cantelon, Mark Harter, T.J. Holowaychuk, Nathan Rajlich, Manning Publications. (Chapters 1-11)
- React Quickly, AzatMardan, Manning Publications (Chapters 1-8, 12-14)

Web Links:

- ExpressJS - <https://www.tutorialspoint.com/expressjs>
- ReactJS - <https://www.w3schools.com/REACT> (and) <https://react.dev/learn#>
- MongoDB - <https://learn.mongodb.com/learning-paths/introduction-to-mongodb>